2024 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW) | 979-8-3503-4479-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICSTW60967.2024.00016

Iterative Optimization of Hyperparameter-based Metamorphic Transformations

Gaadha Sudheerbabu*, Tanwir Ahmad*, Dragos Truscan*, Jüri Vain*†, and Ivan Porres*
*Åbo Akademi University, Turku, Finland
Email: firstname.lastname@abo.fi
† Tallinn University of Technology, Tallinn, Estonia
Email: firstname.lastname@ttu.ee

***Abstract*—Verification and validation of a software system to ensure compliance with the specification and intended functional behaviour often pose a challenge when it lacks an explicit test oracle. We present an efficient black-box metamorphic testing approach in which test cases are automatically generated based on metamorphic transformations. The hyperparameters of several metamorphic transformations are optimized on the fly using a generative AI with a feedback loop for optimal test generation and test suite minimization. The proposed method uses several combined metamorphic relations to define test inputs and to determine the test verdict. The feedback on test quality is evaluated based on the metamorphic relation's fitness function and used to optimize the next iterations of test generation. The effectiveness of the proposed approach is evaluated on an industrial case study of a crane's load position system which lacks an explicit test oracle. The experimental results confirm that optimizing the morphing transformations using the feedback loop improves the effectiveness of metamorphic test input generation. The outcome of the study shows that the approach can be potentially applied for functional safety verification in software systems with a test oracle problem.**

***Index Terms*—Metamorphic testing, Verification and Validation, Artificial Intelligence , Software testing, Generative AI, Test Automation.**

I. Introduction

Functional safety verification and validation [1] of industrial control systems (ICSs) is vital to ensure their operational correctness, as a system failure in such safety-critical applications may result in financial or human life losses. The complexity of such systems and related to that the difficulty of defining a test oracle to determine whether the system behaves as expected often make testing challenging. In the case of ICSs with a vast input space, this problem is even more acute. The three main challenges in designing an optimal test suite for testing such systems are [2] [3]: 1) how to choose the optimal set of test inputs for effective fault detection, 2) how to verify if the system behaviour is correct or not, in the absence of an explicit test oracle, 3) how to measure the adequacy and effectiveness of the tests.

The metamorphic testing (MT) technique introduced by Chen et al. [4] is becoming increasingly popular as a solution to test applications without explicit specification of the test oracle. In MT, some generic properties of the system can be defined by domain experts as metamorphic relations (MRs) between different sets of inputs and their expected outputs. Based on the MRs, an initial set of test inputs, known as *seed inputs*, are either extracted from successful execution logs or generated through any traditional test generation method. A source test case is an initial set of tests performed using seed inputs. The seed inputs are transformed into morphed inputs, using a morphing transformation where both the source and target of the transformation should satisfy the given MR. A MR comprises two parts: an input relation and an output relation [5]. The former specifies the relation between the seed inputs and the morphed inputs, whereas the latter specifies a relation that should hold between the outputs of the seed and, respectively, morphed inputs. The tests performed using seed and morphed inputs are referred to as source tests and follow-up tests respectively.

Metamorphic testing comprises the following steps [5]: a) identify MRs based on system properties extracted from the software requirements specification, b) generate seed input for the source test case based on MRs and execute them, c) generate morphed inputs for follow-up test cases based on MRs and execute them, and d) compare the results of source and follow-up test cases if they satisfy the MRs.

One of the main challenges in MT [6] is automating the morphing process to generate a high-quality set of morphed inputs. To address this challenge, in our previous work [7] we suggested a two-phase metamorphic testing approach, which initially uses random generation to discover the faults, followed by a semi-automatic targeted approach to exploit the identified fault-inducing patterns in generating new inputs for the system. However, for the second phase, there was an open question of how to generate an optimized set of tests based on identified patterns that could cause the system to fail.

To improve the usability, effectiveness, and efficiency of metamorphic testing, in this paper we suggest a more generic approach with two novel contributions:

- We specify several MRs for the same system under test and use them in combination to verify the system. This is based on two observations:
  - In many cases, applying more than one morphing transformation to the seed input improves the variety of morphed inputs (e.g., when testing image processing software the input image can be first rotated, then skewed),
  - Many morphing transformations can be made more uni-

979-8-3503-4479-0/24/$31.00 ©2024 IEEE
DOI 10.1109/ICSTW60967.2024.00016

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 11,2026 at 03:05:54 UTC from IEEE Xplore. Restrictions apply.

versal and easily adjusted using hyperparameters (e.g., rotation angle, skew angle).

- We address the test generation and selection as a multi-objective optimization problem and defer it to a machine learning (ML) algorithm to find the optimal values for the hyperparameters used by morphing transformations. In our case, an optimal value is the one that has a higher chance of creating morphed inputs that will result in the violation of the MRs. Furthermore, we use an online generative approach in which each morphing step is improved incrementally based on the feedback received from the output of the system after each executed test.

The rest of the paper is structured as follows. The methodology of online metamorphic testing using a hyperparameter optimizer and the generic algorithm is presented in Section II. We evaluate our approach in Section III on an industrial system that determines the position of a hanging load attached to the hoisting frame of a crane. We discuss the threats to validity in Section IV and related work in Section V. The paper is concluded in Section VI, where we discuss the plans for future work.

II. Methodology

Main steps of our approach are as depicted in Figure 1. By analyzing the system requirements specification (SRS), we manually extract a set of metamorphic relations that we want to apply to the system under test (SUT) for verification. Each MR has a set of corresponding morphing transformations. The MRs and, consequently, their MTs can be combined to obtain more complex morphed inputs. The morphed output is compared to the seed output using predefined relations specified by each MR. In this work, we assume that each morphing transformation can be characterized by one or several hyperparameters. To obtain the optimal set of morphed inputs, these hyperparameters need to be optimized.

We use an online generative approach to generate the values of the hyperparameters. A fitness function is formulated based on the result of checking the MRs and outputs of the system. The fitness function is used to provide feedback to an optimizer before the next set of morphed inputs is generated. This iterative feedback loop allows to learn and adapt the morphing transformation to generate morphed inputs with a higher chance of violating the MRs.

Consider a system with a seed input $\overrightarrow{S_{in}}$, seed output $\overrightarrow{S_{out}}$, and a set of *n* metamorphic relations $\overrightarrow{MR}$, where each $MR^i = (MR^i_{in}, MR^i_{out})$ consists of an input and an output MR. Each input MR, MR^i_{in} specifies how the seed input is transformed to a morphed input. In our work, each metamorphic transformation is controlled by a set of corresponding hyperparameters, $\overrightarrow{Hp^i}$. The output metamorphic relation MR^i_{out} specifies the relation between the seed output and the morphed output to verify if the outputs satisfy the intended functional behaviour.

Algorithm 1 describes the workflow of applying multiple morphing transformations and optimization steps. The optimization is done using a fitness function based on system outputs and the hyperparameters to generate the morphed input and the test suite. Initially, the hyperparameters for the initial test are randomly generated based on the input MRs from its defined input space (line 1). Then, for each seed input (line 2) the morphing transformations are applied sequentially (line 4) by adding the hyperparameter values derived from the latent space which are then mapped to the input space. The resulting morphed input, M^i_{in} is executed against the SUT, and the morphed output, M^i_{out} is collected (line 7). The verdict is assigned based on the MR check between the seed output and the morphed output of the corresponding test (line 8). The fitness function to calculate the fitness value based on the verdict of the MR check and the system output is invoked in line 9. The hyperparameters are optimized based on the calculated fitness value (line 10) and the test is added to the passed and failed test suites depending on the verdict assigned.

Algorithm 1: Online metamorphic testing with parameter optimization

Input : Seed input: $\overrightarrow{S_{in}}$,
Seed output: $\overrightarrow{S_{out}}$,
Hyperparameters: $\overrightarrow{Hp}$,
Metamorphic relations: $\overrightarrow{MR}$
Output: Passed tests: T_p, Failed tests: T_f

1 $\overrightarrow{Hp} \leftarrow$ Initialize($\overrightarrow{MR_{in}}$, $\mathbb{R}$);
2 **foreach** (S^i_{in},S^i_{out}) **do**
3 **foreach** MR^i_{in} **do**
4 $M^i_{in} \leftarrow$ Generate_Morph($\overrightarrow{Hp}$, S^i_{in}, MR^i_{in});
5 $S^i_{in} \leftarrow M^i_{in}$;
6 **end**
7 $M^i_{out} \leftarrow$ Execute_test(M^i_{in});
8 $verdict^i \leftarrow$ MRCheck(S^i_{out}, M^i_{out}, MR^i_{out});
9 $fitness^i \leftarrow$ Calculate_fitness($verdict^i$, M^i_{out});
10 $\overrightarrow{Hp} \leftarrow$ Optimize($fitness^i$, $\overrightarrow{Hp}$);
11 **if** $verdict^i = true$ **then**
12 $T_p \leftarrow T_p \cup M^i_{in}$;
13 **end**
14 **else**
15 $T_f \leftarrow T_f \cup M^i_{in}$;
16 **end**
17 **end**

The seed output and morphed output are compared in the MR Check step, and a test verdict is assigned based on it. Several types of MR relations, $MR^1_{out}, MR^2_{out}, ..MR^n_{out}$ can be used on the set of outputs, e.g. equality, subset, disjoint, etc, which means satisfiability of their conjunction.

$$MRCheck(S^i_{out}, M^i_{out},\ MR^i_{out}) = MR^1_{out}(S^i_{out}, M^i_{out}) \wedge MR^2_{out}(S^i_{out}, M^i_{out}) \wedge \ldots \wedge MR^n_{out}(S^i_{out}, M^i_{out})$$

The fitness of each test is determined based on its verdict and the outputs. The fitness value or the feedback is used for optimizing the test generation in subsequent morphing transformations.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 11,2026 at 03:05:54 UTC from IEEE Xplore. Restrictions apply.

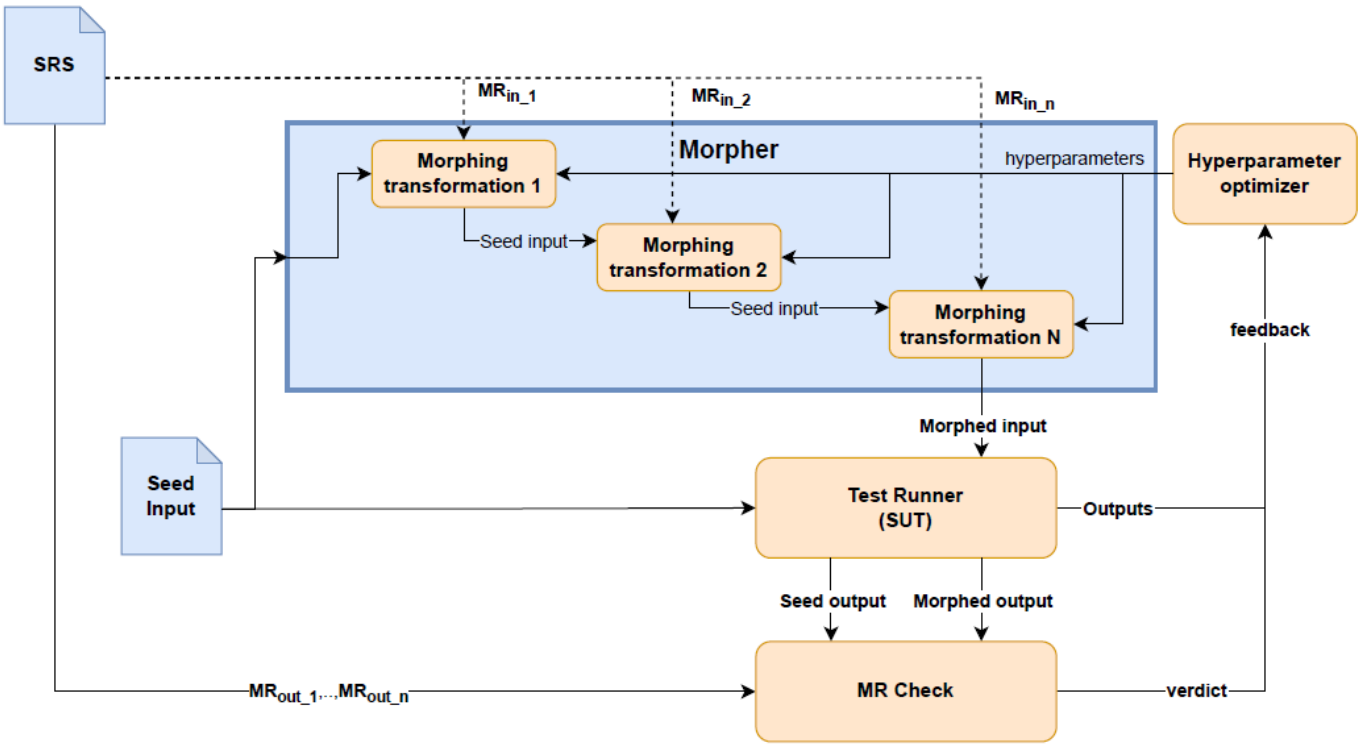


Fig. 1. Overview of the metamorphic testing approach.

III. CASE STUDY AND EVALUATION

We have applied our approach to a Load Position System (LPS), a component of a port crane, which determines the position of a hanging load using attached positional markers on the hoisting frame. The LPS may receive up to 26 markers from a camera module as pixel xy-coordinate pairs. The input may contain three markers on the hoisting frame attached to the load and different light reflections in the environment (water, rain, snow, dust, etc.), which the camera filter could not remove. These reflections captured by the camera module correspond to light reflections in the water while loading containers from the vessel using a Ship-to-Shore (STS) crane. Only the pixel coordinate pairs corresponding to the three positional markers placed on the hoisting frame carrying the load are the *true markers* that determine the position of the load (see Figure 2). The true markers placed on the sides of the hoisting frame are referred to as side markers. The top marker detects the load's tilt and increases the algorithm's probability of identifying all three true markers.

For each input from the camera module comprising an array of pixel coordinate pairs, the LPS tries to identify the true markers and discard the ones corresponding to reflections. Specifically, the main inputs to the LPS system include the number of markers from the camera module, the marker coordinate pairs (their position and size), the position of the trolley, gantry, and hoist relative to camera mounting, and the marker's relative position to the head block's centre.

The LPS produces the following outputs: a Boolean value, $Found_{tm}$, indicating whether true markers are identified, a vector of three integers $[I_{tm}, I_{sm1}, I_{sm2}]$, indicating the index in the input marker array of the positional markers identified as true markers, and the scores indicating the confidence with which the system identified the top marker and the side markers, as the true markers. Whenever the LPS fails to identify the true markers consistently, the system can potentially move to an unsafe state and require human intervention.

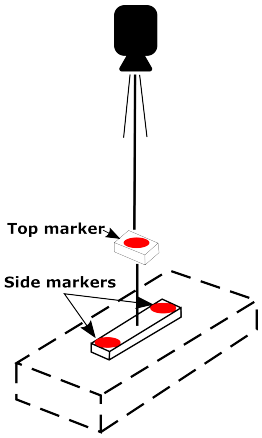


Fig. 2. Positional markers in the LPS

A. Seed input generation

In our approach, we choose the seed input Xs from an input space X as a vector of xy-coordinates X_s =

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 11,2026 at 03:05:54 UTC from IEEE Xplore. Restrictions apply.

$\{s_1, s_2, s_3, \ldots, s_k\}$, where k is the number of seed values and $s_i = [x_1, y_1, x_2, y_2, x_3, y_3]$ corresponds to coordinates of three markers. The seed input for the testing can be extracted from previous execution logs of the system, or generated randomly provided it represents the true data distribution. When the seed input data set is extracted from the execution log, we run an initial test session against the SUT to generate the seed output.

B. Metamorphic relations

The property that we want to check for the LPS is: *"If the system is able to classify correctly a set of positional markers detected by the camera module as true markers in the absence of reflections (noise), it should be able to classify correctly the same positional markers in the presence of reflections when the order of the markers is changed"*. Based on this expected system behaviour, we can extract two MRs:

- MR1: The LPS should identify correctly the true markers in the presence of reflections
  - Seed input: the set of true markers, denoted as X_s,
  - Input MR: MR^1_{in} :$X_{m1} = X_s \frown X_n$, is a concatenation of X_s and X_n, where X_n is the vector $[x_1, y_1, x_2, y_2, \ldots, x_n, y_n]$ selected from the input space comprising of the coordinate pairs corresponding to reflection markers which we denote as noise coordinate pairs.
  - MT1: select and append noise coordinate pairs to the seed input MR^1_{in} :$X_{m1} = X_s \frown X_n$, where X_{m1} is the output of the first morphing transformation obtained by selecting a specified number of noise coordinate pairs from the input space $X_n = SELECT(I, n)$ and appending it to seed input, X_s; $X_{m1} = APPEND(X_s, X_n)$.
  - Output MR1: $MR^1_{out} : Y_{m1} == Y_s$, where Y_{m1} and Y_s are the output of the LPS when executing the morphed input X_{m1} and, respectively, the seed input X_s.
- MR2: The LPS should identify correctly the true markers regardless of their order in the input
  - Seed input: the set of true markers and noise coordinate pairs, X_{m1} as generated by MT1
  - Input MR: $MR^2_{in} : X_{m2} = PERMUTE(X_{m1})$
  - MT2: change the order of the seed input X_{m1} via permutation to generate X_{m2}
  - Output MR2: $MR^2_{out} : Y_{m_1} == Y_{m_2}$, where Y_{m1} and Y_{m2} are the outputs when executing the seed X_{m1} and the morphed input X_{m2} respectively .

Consequently, the combined output relation that has to be checked is $MR_{out} \equiv MR^1_{out} \wedge MR^2_{out}$, which in this case corresponds to evaluating $MR_{out} \equiv Y_s == Y_{m2}$.

C. Hyperparameter selection

The two morphing transformations defined above imply two operations *select* and *permute*, which rely on random number generators to accomplish their task. For guiding the optimization of the morphed input generation, we expose the random seed of the permutation operator as a hyperparameter of each operation as follows: $SELECT(I, n, rand_seed_s)$ and $PERMUTE(X_{m1}, rand_seed_p)$ (see Figure 3).

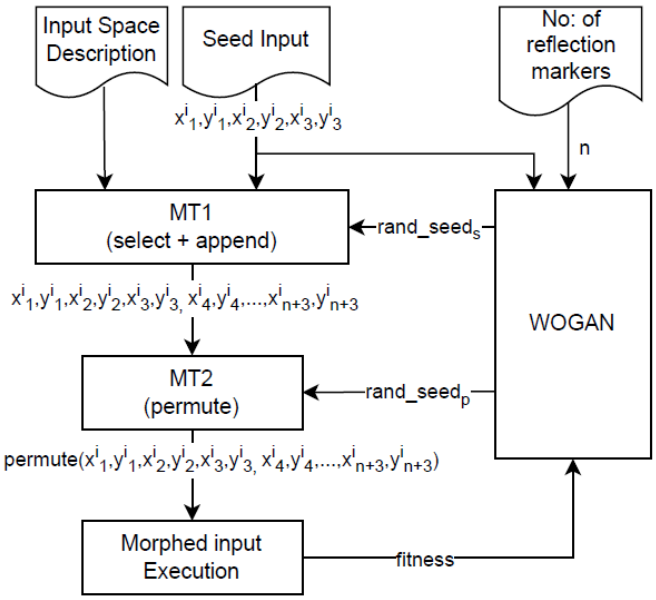


Fig. 3. Overview of the optimization process using WOGAN

In our case, $rand_seed_s$ is a vector of $2n$ values between [-1, 1], in the form $[rs_{x1}, rs_{y1}, rs_{x2}, rs_{y2}, \ldots, rs_{xn}, rs_{yn}]$, where n is the number of noise coordinate pairs to be selected from the input space. Using the same random seed for each iteration would select the same coordinate from the input space. As such, we address the selection of the random seeds for selection and permutation operations as a multi-objective optimization problem which will provide test inputs that are effective at revealing faults.

In this work, we utilize the Wasserstein Generative Adversarial Network (WOGAN)-based algorithm proposed in [8]. It uses the Wasserstein distance to stabilize the gradient during the training process, thereby improving the quality of the generated tests. In previous experiments on test generation [9] the algorithm proved that it can generate efficiently a diverse set of tests. Initially, WOGAN generates random values for the seed values, but after a given threshold, it begins to learn from the fitness function which of the previously generated values was more effective at creating failing morphed inputs. During this interactive process, WOGAN optimizes for both the selection and the permutation of the random seed values. However, we would like to point out that our approach can similarly utilize other optimization methods (such as genetic algorithms) to find the optimal HP values for MRs. Future work will investigate the effects of different optimization methods on the performance of our approach.

D. Morphing transformations

During an actual execution, the LPS receives a continuous trace corresponding to the positional markers from the camera module. In our morphing transformations, we add random noise points to a spatially continuous trace of seed inputs

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 11,2026 at 03:05:54 UTC from IEEE Xplore. Restrictions apply.

collected from the actual execution logs. The noise coordinate pairs generated are appended to the seed input in this first step of morphing transformation.

Morphing transformation 1 takes n, the number of noise point sets to be generated, as an input provided by the user and a random seed ($rand_seed_s$) vector provided by the hyperparameter optimizer which is used for selecting the points from the input space. It generates a set of input coordinates in the form of the vector of pairs $[x_1, y_1, x_2, y_2, \ldots, x_n, y_n]$, with x_i, x_j in the range [0,131072]. The noise data generated are appended to the seed input in this first step of morphing transformation resulting in a set of input coordinates.

In the case of morphing transformation 2, the order of the input coordinate pairs is shuffled using the seed value $rand_seed_p$. We run an initial offline test session against the SUT to collect the seed output and index values of the actual seed input after the shuffle is used for the MR check for follow-up tests. Morphing transformation 2 simply creates a permutation of the input markers received from the MT1.

E. MR Check

To determine the verdict of the online test executed on the SUT, we compare the outputs of the seed input against the outputs of the morphed input with respect to the positions of the identified markers in the system outputs using the relation:

$$Found_{tm} \wedge I_{tm} = tm_{index} \wedge I_{sm1} = sm1_{index} \wedge I_{sm2} = sm2_{index} \quad (1)$$

where $Found_{tm}$ and $[I_{tm}, I_{sm1}, I_{sm2}]$ are the outputs of the system that indicate if the true markers and their index positions are identified. These are compared against the index position of the seed markers $tm_{index}, sm1_{index}, sm2_{index}$ after the morphing transformation 2.

Any test that violates the MR is considered a failed test, indicating that the SUT failed to identify the true markers. Figure 4 shows an example of missed classification of markers in the presence of noise. Since two of the noise points *M2, M5* follows the same geometrical pattern of the side markers *M1, M4*, the system fails to identify *M1, M3, M4* as true markers.

F. Fitness function

Based on the *MR Check* (in subsection III-E), feedback to the optimizer is formulated as a fitness function with its value in the range [0,1]. In the case of the LPS, for the passed tests (where the index positions match), the scores from the system for the top marker and the side markers are used to define the fitness function as follows:

$$Fitness = \begin{cases} \dfrac{(s_{top} + s_{side}) - min_{score}}{(max_{score} - min_{score})}, & MR_{out}\ satisfied \\ 0, & MR_{out}\ violated \end{cases} \quad (2)$$

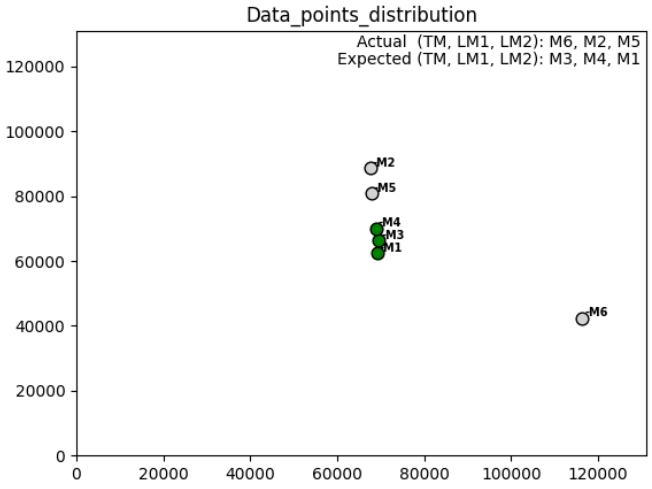


Fig. 4. Seed and noise data distribution of a failing test

In the fitness function formula (2), s_{top} and s_{side} are the outputs of the system, max_{score} is the sum of the maximum score value that the system can assign to top and side markers, and min_{score} is the minimum possible score.

It has to be noted that the fitness function is assigned a value zero for the failed tests, and the learned model targets to minimize the fitness function by tuning the hyperparameters. These scores indicate the confidence with which the system identifies the markers as true markers. As per the system design, it assigns gold and silver scores for the two selected top markers and side markers with a value in the range [0,100]. In case of a passed test, output scores for the top and side markers from the SUT are summated. This combined score value will be in the range [0,200], then normalized to the range [0,1], as the feedback fitness. These values are sent to the hyperparameter optimizer as feedback from the MR check to improve the quality of the test input generation.

G. Tool support

Except for the formalization of the MR, the approach is fully automated (see Figure 5). The tool takes as input a file containing the seed input values and the number of noise coordinate pairs to be generated in each session. Both the WOGAN tool[1] and the morphing transformations are written in Python and the Python testing framework `Pytest` is used to implement the test adaptation.

The PLC application program runs on CODESYS SoftPLC V3.5 with an in-built OPC UA server that enables the communication between the CODESYS OPC UA server [10] and the Python OPC UA client [11] via the OPC UA [12] data exchange protocol.

H. Results

We have conducted three sets of experiments to evaluate the effectiveness of the proposed approach in morphed input generation and test suite minimization.

[1]based on the implementation available at https://gitlab.abo.fi/stc/stgem

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 11,2026 at 03:05:54 UTC from IEEE Xplore. Restrictions apply.

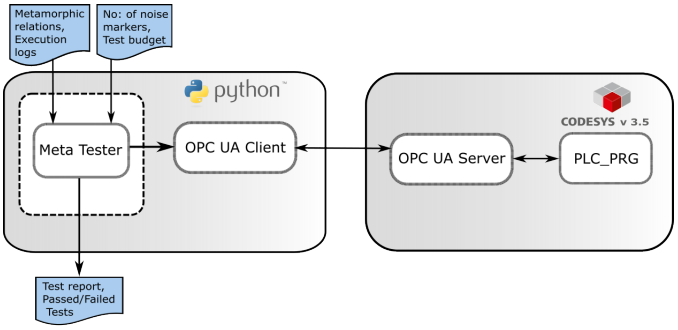


Fig. 5. Tool support for metamorphic testing of LPS

The seed input is selected from execution logs of the system collected at operations. As an alternative, the seed input can be generated from simulators. In our experiments, the seed input contains a continuous sampling trace of 600 readings of the true marker coordinates taken for 300 seconds. Each seed input reading contains exactly three marker coordinates, corresponding to true markers. An initial source test session against SUT was conducted to confirm that the system identifies the true markers correctly.

The morphed input is created by applying the previously defined morphing transformations. Each set of true marker coordinates is complemented with noise coordinate pairs and permuted. Therefore, the generated morphed input will have the same number of tests as the seed input, and in order to increase the number of generated tests, we replicate the seed input trace as needed for the desired test budget.

For the experiments, we create a seed input of 4000 tests and run test sessions that will generate 1, 2, and respectively, 3 noise coordinate pairs for each true marker pair in the coordinates. For statistical significance, we repeat each test session 20 times. For each test session, we assign 20% (800 tests) of the total test budget for the random exploration and 80% (3200 tests) for the training phase.

The test verdict is assigned based on the MRs which expect the system to identify correctly the markers in the seed input as the true markers.

For each experiment, we collect and calculate the following metrics:

- number of failed tests, that violated the MR,
- number of passed tests, that did not violate the MR,
- fault detection rate (FDR) - as the number of tests that revealed a fault divided by the total number of generated tests in the test session,
- the fitness value calculated after each test,
- test execution time for each session.

Figure 6 depicts the cumulative number of failed tests versus the number of generated tests for each experiment. As one can notice, the figure shows a clear distinction between the random phase and the learning phase, in which the test generator is able to generate an almost linearly increasing number of fault-revealing tests. Therefore, our hyperparameter optimization approach shows a clear benefit compared to using the random generation for the entire test session. Furthermore, the number of failing tests rapidly increases with the number of noise coordinate pairs, which shows the SUT's increasing difficulty in classifying the markers correctly. This difference can also be observed from the number of failed tests in Table I, where the mean value for each set of experiments is calculated as the overall mean of arithmetic means obtained from each test session in that corresponding set.

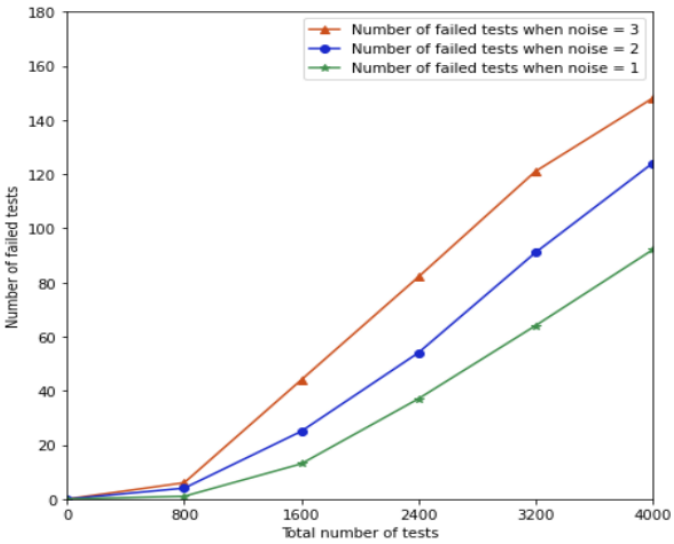


Fig. 6. Comparison of the cumulative number of failed tests found in the random phase (initial 800 tests) and the learning phase (next 3200 tests) of the test session by different values of noise inserted in seed input.

We would also like to point out that the mean FDR for each experiment (as displayed in Table I) is roughly similar to the one obtained in our previous work [7], [13] during the semi-automatic exploitation phase. To find the fault patterns that direct the test generation process in the exploitation phase, however, a large test budget was expended during the exploration phase. In comparison, test generation is fully automated and fault patterns can be identified within a limited test budget using iterative optimization utilizing hyperparameters. From this, we can conclude that the WOGAN optimizer was able to learn good values for the hyperparameters similar to the human test designer.

To reflect the capacity of the tool to optimize the hyperparameter values, we calculate the mean fitness of the final 80% and, respectively, of the final 20% of each experiment (Table I). As one can notice, the mean fitness value decreases slightly from the random phase to the final 80% set and then to the final 20%. Also, the standard deviation of the fitness values in the final 80% and 20% shows that the generated morphed inputs consistently reduced the fitness level closer to the observed mean. This shows that the tool is able to learn to optimize the parameters, however, this ability is limited to a certain extent. This can be explained by the fact that the system under test is a system that has been tested extensively using traditional testing methods and the probability of finding

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 11,2026 at 03:05:54 UTC from IEEE Xplore. Restrictions apply.

new faults was quite low. On the other hand, it shows that our approach can be seen as complementary to other test methods to find additional faults that have not emerged with standard testing methods.

Based on the analysis done on the failing tests, it was observed that the system has a limitation that causes an incorrect classification or a missed classification of true markers in the presence of certain types of noise. Any noise input pattern triggering an incorrect classification is considered a fault-inducing pattern. It is also observed that a noise point that overlaps or lies close to the seed input can trigger an incorrect classification. Additionally, two or three noise coordinate pairs that follow the same geometrical pattern as the seed coordinate pairs can potentially trigger faulty detection and is considered as system limitation. Moreover, increasing the number of noise coordinate pairs to be generated increases the probability of triggering the system limitation and results in more failing tests during the initial random search, thereby providing more training data for the hyperparameter optimizer.

To verify this assumption, we have executed a fourth experiment, where we set the number of marker coordinate pairs to be sent to the SUT as a hyperparameter for optimization and chose a budget of 300 tests per session. In these test sessions, initially, the number of inserted markers was picked randomly from the range [1,23], and it was observed that during the learning phase, the most often picked value was the maximum value from the given range, i.e., 23, which triggered the faulty detection system limitation. As summarised in Table II, the fitness value during the final 80% and final 20% dropped to 0.12 and 0.1, respectively, in the fourth set (Noise 1-23). This indicates that as the system continues to receive the noise towards the maximum limit in the training phase, it fails to identify the true markers with a sharp decline in the observed fitness value.

Figure 7 shows the fitness values observed in the four test sets over 20 test sessions per set, where the input parameter is the number of noise coordinate pairs to be inserted for evaluating the robustness of the SUT. To conclude, the approach effectively minimized the target fitness value of the system with an optimizer integrated into the metamorphic testing process before the morphing transformation phase.

IV. Threats to validity

The current work has not been validated against time-critical systems. Our experiments are applied to a PLC application written in FBDs and structured text with no timer functions. It is possible that the approach may collect less accurate execution information in the presence of time-dependent variables. However, the system in our study is a real-world ICS in PLC programming language for which the proposed method using OPC UA as a communication protocol is standardized under Industry 4.0 for its technical interoperability [14].

The effectiveness reported in the study is based on the chosen MR for the system under test. The results reported from the experiments are based on the input variables defined in the input MR and output behaviour, and the test verdict

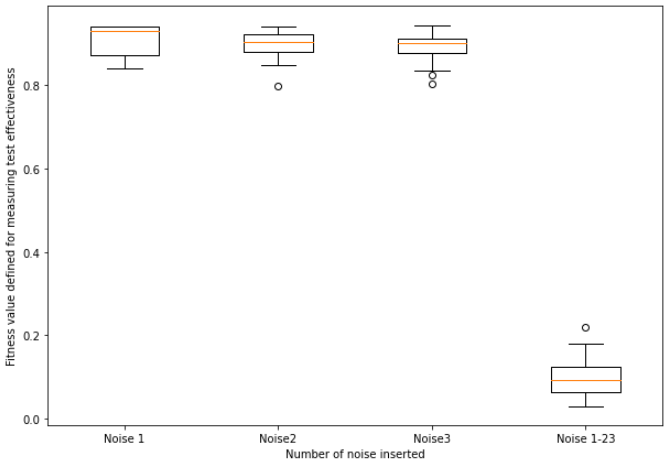


Fig. 7. Box plots for mean fitness values observed in three different test sets over 20 test sessions per set.

relies on those. Like any other testing method, the MT is not exhaustive, even if the MR has been identified. For ICSs with large input space, different MRs can be identified. The technique applies to MRs, which involve input parameters with numerical values to be selected from an input space. The choice of MRs that use a different set of input variables may influence the outcome of the metamorphic testing. Therefore, the approach's effectiveness for any industrial software system with a test oracle problem may vary depending on the MRs that can be defined for it and the performance of the parameter optimization algorithm selected.

V. Related work

The metamorphic testing method has been previously applied to test Deep Learning (DL) based systems for image classification, image recognition, and autonomous driving systems. DeepBackGround [15], an MT tool for the robustness evaluation of image recognition systems based on deep learning used domain-specific MRs. The tool uses Background-Relevance (BRC) as a metric to verify the influence of the image's background region on the recognition system's outcome. Like our work, their tool derived follow-up tests for MT by adding noise to the background region of the actual image to test the image recognition accuracy of DL systems used in safety-critical domains. However, our work uses the optimizer for improving the metamorphic test input generation to falsify the system based on the MR-violation feedback.

An iterative metamorphic testing [16] approach by Wu, P et al. proposed to apply metamorphic relations in a chain style to improve the fault detection capabilities of metamorphic testing. An on-the-fly metamorphic testing where MR sequences were derived dynamically during the test procedure and the effectiveness was estimated using mutation analysis technique by measuring the mutation score and fault detection ratio. Our proposed method improves the test generation by optimization of hyperparameters via a feedback loop and multi-step morphing transformation. We evaluate the effec-

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 11,2026 at 03:05:54 UTC from IEEE Xplore. Restrictions apply.

No. of noise coordinate pairs	Test Budget	Mean number of failed tests	Mean FDR	Mean fitness random%	Mean fitness final 80%	Mean fitness final 20%	SD fitness final 80%	SD fitness final 20%
1	4000	86.1	0.02	0.94	0.91	0.90	0.02	0.03
2	4000	126.85	0.03	0.94	0.90	0.89	0.02	0.03
3	4000	150.95	0.032	0.93	0.89	0.88	0.02	0.03

TABLE I
TEST EXECUTION RESULTS SUMMARY

No. of noise coordinate pairs	Test Budget	Mean number of failed tests	Mean FDR	Mean fitness Random	Mean fitness final 80%	Mean fitness final 20%	SD fitness final 80%	SD fitness final 20%
1-23	300	229.1	0.763	0.6	0.12	0.1	0.03	0.05

TABLE II
TEST EXECUTION RESULTS SUMMARY, EXPERIMENT 4

tiveness of our approach in terms of its capability to generate tests that minimize the fitness function defined for the SUT. Metamorphic testing has been used to verify the robustness of GANs [17] and DL-based image recognition systems [18]; however, to the best of our knowledge, metamorphic testing with an online feedback-driven approach using WOGAN to test an industrial control system is a novel attempt.

VI. CONCLUSION

We have presented an approach for metamorphic testing using an online test generator for optimizing the test input generation with a feedback loop. This work uses Wasserstein Generative Adversarial Networks (WOGAN)-based algorithm to optimize the hyperparameters for morphing transformation. It is to be noted that our approach can utilize other optimization methods (such as Genetic algorithms) to find the effective hyperparameter values for the MRs. Using WOGAN to drive the test generation allows us to fully automate the test generation and execution process compared to our previous approach which was semi-automated. Future work will investigate the effects of different optimization methods on the performance of our approach. The technique has been successfully applied for functional safety verification of an industrial case study with a test oracle problem. The results based on the online testing conducted on the SUT showed that using the feedback loop for optimizing the test input generation could successfully minimize the target fitness and generate tests that can reveal faults in the system within a limited test budget.

ACKNOWLEDGMENT

This work has received funding from Business Finland under the Virtual Sea Trial project and from the European Union under projects VeriDevOps (Horizon 2020 GA 957212) and AIDOaRT (ECSEL GA 101007350).

We would also like to thank Dr. Jarkko Peltomäki for his technical advice on integrating the WOGAN tool into our approach.

REFERENCES

[1] A. Corso, R. Moss, M. Koren, R. Lee, and M. Kochenderfer, "A survey of algorithms for black-box safety validation of cyber-physical systems," *Journal of Artificial Intelligence Research*, vol. 72, pp. 377–428, 2021.

[2] A. Bertolino, "Software testing research: Achievements, challenges, dreams," in *Future of Software Engineering (FOSE'07)*. IEEE, 2007, pp. 85–103.

[3] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE transactions on software engineering*, vol. 41, no. 5, pp. 507–525, 2014.

[4] Y. T. Chen *et al.*, "Revisiting the relationship between fault detection, test adequacy criteria, and test set size," in *Proceedings of the 35th IEEE/ACM Intl. Conference on Automated Software Engineering*, 2020, pp. 237–249.

[5] H. Liu, X. Liu, and T. Y. Chen, "A new method for constructing metamorphic relations," in *2012 12th international conference on quality software*. IEEE, 2012, pp. 59–68.

[6] T. Y. Chen *et al.*, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys (CSUR)*, vol. 51, no. 1, pp. 1–27, 2018.

[7] G. Sudheerbabu, T. Ahmad, F. Sebek, D. Truscan, J. Vain, and I. Porres, "A two-phase metamorphic approach for testing industrial control systems," in *2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2022, pp. 1–4.

[8] J. Peltomäki, F. Spencer, and I. Porres, "Wasserstein generative adversarial networks for online test generation for cyber physical systems," in *Proceedings of the 15th Workshop on Search-Based Software Testing*, ser. SBST '22. New York, NY, USA: ACM, 2023, p. 1–5. [Online]. Available: https://doi.org/10.1145/3526072.3527522

[9] J. Peltomäki, F. Spencer, and I. Porres, "WOGAN at the SBST 2022 CPS Tool Competition," in *Proceedings of the 15th Workshop on Search-Based Software Testing*, ser. SBST '22. New York, NY, USA: ACM, 2023, p. 53–54. [Online]. Available: https://doi.org/10.1145/3526072.3527535

[10] "CODESYS OPC UA," Feb. 2020. [Online]. Available: https://www.codesys.com/products/codesys-runtime/opc-ua.html

[11] O. Roulet-Dubonnet, "Python OPC-UA Documentation," 2021. [Online]. Available: https://python-opcua.readthedocs.io/en/latest/index.html

[12] S.-H. Leitner and W. Mahnke, "OPC UA–service-oriented architecture for industrial applications," *ABB Corporate Research Center*, vol. 48, no. 61-66, p. 22, 2006.

[13] G. Sudheerbabu, T. Ahmad, D. Truscan, and J. Vain, "Metamorphic testing for verification and fault localization in industrial control systems," in *CyberSecurity in a DevOps Environment: From Requirements to Monitoring*. Springer, 2023, pp. 127–159.

[14] M. Schleipen *et al.*, "Opc ua & industrie 4.0-enabling technology with high diversity and variability," *Procedia Cirp*, vol. 57, pp. 315–320, 2016.

[15] Z. Zhang *et al.*, "Deepbackground: Metamorphic testing for deep-learning-driven image recognition systems accompanied by background-relevance," *Information and Software Technology*, vol. 140, p. 106701, 2021.

[16] P. Wu, "Iterative metamorphic testing," in *29th Annual International Computer Software and Applications Conference (COMPSAC'05)*, vol. 1. IEEE, 2005, pp. 19–24.

[17] H. Park *et al.*, "Robustness evaluation of stacked generative adversarial networks using metamorphic testing," in *2021 IEEE/ACM 6th International Workshop on Metamorphic Testing (MET)*. IEEE, 2021, pp. 1–8.

[18] Y. Torikoshi *et al.*, "Sensitive region-based metamorphic testing framework using explainable ai," *arXiv preprint arXiv:2303.07580*, 2023.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 11,2026 at 03:05:54 UTC from IEEE Xplore. Restrictions apply.